

TLS/SSL Issues Runbook

Last updated: 2026-06-08

Author: Henry Post

Links

[Adapted from this guide.](#)

Purpose

This runbook explains how to resolve SSL/TLS certificate validation failures caused by tools that do not automatically trust the NetSkope root certificate.

Use these steps when Git, npm, Node.js, Python, Java, Azure CLI, AWS CLI, Snowflake clients, Cursor, Claude Code, StackHawk, Gradle, or similar developer tools fail with certificate validation errors.

Recommended Approach

Prefer fixes in this order:

1. Use the operating system certificate store when the tool supports it.
2. Configure the tool to trust the NetSkope root certificate file.
3. Pin the NetSkope certificate only when the tool cannot use the operating system certificate store.

Do not disable certificate validation with flags such as `--ignore-ssl`, `--insecure`, `strict-ssl=false`, or equivalent settings. Those options hide the immediate error but leave the tool vulnerable to TLS interception and impersonation.

On managed Windows workstations, the NetSkope certificate should normally exist here:

```
C:\ProgramData\netskope\stagent\data\nscacert.pem
```

NetSkope Root CA Certificate

If you need to create a local certificate file manually, save the certificate below as `netskope-root.crt` or `netskope-root.cer`. Include both the `BEGIN CERTIFICATE` and `END CERTIFICATE` lines.

```
-----BEGIN CERTIFICATE-----
MIIE9zCCA9+gAwIBAgIJAJJDI3J7eaSmMA0GCSqGSIb3DQEBBQUAMIGtMQswCQYD
VQQGEwJVUzETMBEGA1UECBMKQ2FsaWZvcn5pYTESMBAGA1UEBxMjTG9zIEFsZG9z
MRUwEwYDVQKKEwxuZXRTa29wZSBjb20wZS5jb20xJTAjBgkqhkiG9w0BCQEW
dDEdMBsGA1UEAxMYU2FhZG1pb29wZS5jb20wZS5jb20xJTAjBgkqhkiG9w0BCQEW
FmNlcnRhZG1pb29wZS5jb20wZS5jb20wZS5jb20wZS5jb20wZS5jb20wZS5jb20w
MjMyMTE3WjCBTELMAKGA1UEBhMCVVMxZS5jb20wZS5jb20wZS5jb20wZS5jb20w
BgNVBACTCUxvcmVudC5hbnRvczEVMGMGA1UEChMMbWV0U2tvcGUgSW5jMRgwFgYDVQQL
Ew9DZXJ0IE1hbmFnZW11bnQxHTAbBgNVBAMTFGNhYWRtaW4ubmV0c2tvcGUuY29t
```

```
MSUwIwYJKoZIhvcNAQkBfHjZjZXJ0YWRtaW5AbmV0c2tvcGUuY29tMIIBIjANBgkq
hkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAXzKWllmAAcY6P8qG10jPrs4/5b/ofQX
e7Gd/vCoAtWEC1Q6TxClYkU17sYF9bu1CUD3NHWSQfReezCKGyBPafcu+twg+Dcd
yc/uaCxz6V9BpsPT6lgzYkDDpVE63I9z1MGnaS+hIAdplaPjcXHUXe/mYNXtKAtE
duaUDg76R1588BLvOVWn6txFoDxKqDsFkzAoI3uoP07AH/0EFAokAYzt9f1G1hLE
v30U9feQ00IggSpdiqo/pLdaRUaAI2Zm0fm5DNu/6vEF0G6p7rNY1rIUynTVmzKY
P+5w61fsBN190UYCvCgF6N0pBZgl93xCocnZ1Zd3DjR9M2QFFdXrXwIDAQABo4IB
FjCCARIwHQYDVR00BBYEFNQ+GYzg+VGt/zU04p1Ja1DuzKF+MIHiBgNVHSMEgdow
gdeAFNQ+GYzg+VGt/zU04p1Ja1DuzKF+oYGzpIGwMIGtMQswCQYDVQQGEwJVUzET
MBEGA1UECBMKQ2FsaWZvcm5pYTESMBAGA1UEBxMjTG9zIEFsdG9zMRUwEwYDVQQK
EwxuZXRTa29wZSBjb29wZSB5b20xJTAjBgkqhkiG9w0BCQEWFmN1cnRhZG1p
bkBuZXRza29wZSB5b22CCQCSQyNye3mkpjAMBGNVHRMEBTADAQH/MA0GCSqGSIb3
DQEBBQUAA4IBAQCkwspwM/SpIrbFEKSh4bfhR6p9YZ8nL6V5Q68lQooIFBTndPi4
0lazoss4NrUchuCvDe9LHgFDQMf5LABiyy6RixMhWYa85hoUHKULDwunSYHJEKKh
OzuxQ131m7/jQtL7RtAvTzdneykI51rGd0jvCgxFDa6lBS5fmDUs+5DpDbHWamExC
ksBmuDhV8+yh+7MZriSOCEzDNlxUm8qbK2TYvaQhM7n+YM4BG+2DVWtVPtyiVTFd
ss27AwG2hyhc+Czg5PU2V1Z+JsQyE014G8xLHH6hdQPWC8gBlsPc/nemEBxhnqNi
fAkUAvHXsEwI5PdVcvP2KI4hSz60FQNMQEr6
-----END CERTIFICATE-----
```

Install the Certificate in Operating System Trust Stores

Debian and Ubuntu

Copy the certificate to the local CA certificate directory, then update the trust store:

```
sudo cp netskope-root.crt /usr/local/share/ca-certificates/
sudo update-ca-certificates
```

RHEL and Fedora

Copy the certificate to the CA trust anchor directory, then update the trust store:

```
sudo cp netskope-root.crt /etc/pki/ca-trust/source/anchors/
sudo update-ca-trust
```

Windows

The NetSkope certificate should be pre-installed on managed Windows workstations. To inspect or add certificates manually:

1. Press **Win+R**.
2. Run **certmgr.msc**.
3. Check the trusted root certificate stores for the NetSkope certificate.

Tool-Specific Fixes

Getting a certificate using `openssl` command line on Linux

To get a root (or closest to root) cert on Linux, using only `openssl` command line, follow these steps:

```
openssl s_client -showcerts -connect example.com:443 </dev/null 2>&1 >mycert.crt
```

Replace `example.com` with the host you're trying to connect to.

You then need to edit `mycert.crt` with `nano` or a similar text editor.

Remove all the extra stuff that's not in `---ASCIIARMOR---` format.

Keep the certificate with the highest index (number) printed next to it. `0` is the leaf cert, `1` is an intermediate (or root cert), etc.

Then, copy it using this guide into the directory that stores certificates and run `update-ca-certificates` or whatever Distro-specific command applies.

The resulting `mycert.crt` file should look like this:

```
-----BEGIN CERTIFICATE-----  
(...data...)  
-----END CERTIFICATE-----
```

Git Clone on Windows Fails Over HTTPS

Configure Git for Windows to use the Windows certificate store:

```
git config --global http.sslBackend schannel
```

Restart the shell and retry the clone.

npm, Node.js, Cursor, and Claude Code

Set `NODE_EXTRA_CA_CERTS` to the NetSkope certificate path:

```
setx NODE_EXTRA_CA_CERTS "C:\ProgramData\netskope\stagent\data\nscacert.pem"
```

Restart the shell, IDE, terminal, or service after setting the variable.

Use this same approach for Node-based tools, including npm, Cursor extensions, Claude Code CLI/TUI, and other tools that run on Node.js.

Azure CLI

Microsoft recommends setting `REQUESTS_CA_BUNDLE` when working behind a proxy:

```
setx REQUESTS_CA_BUNDLE "C:\ProgramData\netskope\stagent\data\nscacert.pem"
```

Restart the shell before rerunning Azure CLI commands.

Reference: [Troubleshoot Azure CLI behind a proxy](#)

AWS CLI

Set `AWS_CA_BUNDLE` to the NetSkope certificate path:

```
setx AWS_CA_BUNDLE "C:\ProgramData\netskope\stagent\data\nscacert.pem"
```

Restart the shell before rerunning AWS CLI commands.

Python on Windows

Prefer using Windows system certificates:

```
pip install pip-system-certs
```

This is preferred over manually pinning a certificate file.

If you use `pip-system-certs`, remove certificate-pinning environment variables that may override the Windows certificate store:

```
[Environment]::SetEnvironmentVariable("REQUESTS_CA_BUNDLE", $null, "User")  
[Environment]::SetEnvironmentVariable("SSL_CERT_FILE", $null, "User")
```

Restart the shell after changing environment variables.

Python Certificate Pinning

If Windows system certificates are not available, save the NetSkope certificate to a local path such as:

```
C:\Tools\netskope-root.cer
```

Then set both Python-related certificate environment variables:

```
setx REQUESTS_CA_BUNDLE "C:\Tools\netskope-root.cer"
```

```
setx SSL_CERT_FILE "C:\Tools\netskope-root.cer"
```

For a single `requests` call, you can also pass the certificate path directly:

```
import requests

response = requests.get(
    "https://bayview.com",
    verify=r"C:\Tools\netskope-root.cer",
)

print(response.status_code)
```

Python 3.13 and Newer Basic Constraints Errors

Newer runtimes such as Python 3.13 and recent Azure CLI versions enforce RFC 5280 more strictly. A certificate may fail validation if the Basic Constraints extension is not marked critical or if required Key Usage fields are missing.

You can inspect a certificate with OpenSSL:

```
openssl x509 -in caadmin.netskope.com.crt -text -noout
```

The current NetSkoPe certificate at the top of this guide includes the vendor fix. If the error persists after updating the certificate file, contact Henry Post and Jon Juhler.

Reference: [OpenSSL X.509 validation source](#)

Java and StackHawk

Preferred approach: use the Windows certificate store.

```
setx JAVA_TOOL_OPTIONS "-Djavax.net.ssl.trustStoreType=Windows-ROOT -Djavax.net.ssl.trustStore=NONE"
```

Restart the shell, IDE, scanner, service, or build agent after setting the variable.

If using MacOS, you can use this environment variable:

```
export JAVA_TOOL_OPTIONS="-Djavax.net.ssl.trustStoreType=KeychainStore -Djavax.net.ssl.trustStore=NONE"
```

Java Certificate Pinning

If Java cannot use the Windows certificate store, import the NetSkope certificate into the Java trust store. Update the paths for the installed JDK or JRE:

```
keytool.exe -import -trustcacerts -alias netskope -file  
"C:\ProgramData\netskope\stagent\data\nscacert.pem" -storetype JKS -keystore  
"C:\Program Files\Java\jdk-25\lib\security\cacerts"
```

The default Java trust store password is commonly `changeit`, unless the local installation has changed it.

Gradle

Configure Gradle JVM arguments to use the Windows certificate store:

```
jvmArgs = [  
    "--add-opens=java.base/java.nio=ALL-UNNAMED",  
    "-Djavax.net.ssl.trustStoreType=Windows-ROOT",  
    "-Djavax.net.ssl.trustStore=",  
    "-Djavax.net.ssl.trustStorePassword=",  
]
```

Snowflake Connection Testing

A Snowflake connection test script is available for diagnosing connection failures and printing detailed SSL information.

Ask Henry Post on Microsoft Teams for the script, or use the Bitbucket copy if you have access:

```
https://bitbucket.org/bayview-asset-  
management/appsecautomation/src/master/scripts/TestSnowflake/TestSnowflake.py
```

Export a CA Certificate from Chrome or Island

If you need to export a certificate manually:

1. Open the affected site in Chrome or Island.
2. View the site certificate from the browser security or certificate details panel.
3. Export the root or intermediate certificate as Base-64 encoded X.509.
4. Save the file with a `.cer` or `.crt` extension.
5. Use the saved certificate path with the relevant tool-specific setting above.

Desktop Engineering Automation

When Desktop Engineering packages developer tools in Tanium or the Bayview App Store, the package should include a post-install step that trusts the NetSkope certificate:

```
C:\ProgramData\netskope\stagent\data\nscacert.pem
```

Examples:

```
setx NODE_EXTRA_CA_CERTS "C:\ProgramData\netskope\stagent\data\nscacert.pem"
```

```
C:\path\to\keytool.exe -import -trustcacerts -alias netskope -file  
"C:\ProgramData\netskope\stagent\data\nscacert.pem" -keystore  
"$env:JAVA_HOME\lib\security\cacerts" -storepass changeit
```

The exact command differs by package manager or runtime, but the goal is the same: install developer tooling so it trusts the NetSkope CA by default.

Recommended long-term process:

1. Desktop Engineering bundles developer tools such as JDK/JRE, Python, pip, conda, Maven, Gradle, npm, and Node.js through Tanium or the Bayview App Store.
2. AppSec and developer teams provide the certificate trust command or environment variable required by each tool.
3. Desktop Engineering includes the trust configuration as a post-install step.
4. Developers should not need to disable TLS validation or manually add the NetSkope certificate for standard tools.

This should reduce Desktop Engineering tickets by making software installs self-service and preventing recurring SSL/TLS failures during common commands such as `pip install`, `npm install`, `git clone`, and scanner setup.